

Trabalho I: Exploração de Estratégias de Alocação de Dados e Threads

Matheus Tregnago

8 de junho de 2026

1 Introdução

Quando um algoritmo é paralelizado com OpenMP, não basta apenas distribuir o trabalho entre várias *threads*. A forma como as *threads* são posicionadas nos cores do processador e a forma como os dados são alocados na memória também influenciam bastante o desempenho final. Em máquinas modernas, que possuem vários processadores e mais de um nó de memória, como a arquitetura NUMA, diferentes configurações influenciam diretamente o tempo de execução de um programa.

O objetivo deste trabalho é investigar como diferentes estratégias de alocação de *threads* e de dados afetam o desempenho dos programas em OpenMP. Para isso, foi utilizado o cálculo do conjunto de Mandelbrot, uma aplicação *compute-bound*, e o Stencil 2D de Jacobi, que é *memory-bound*. Os experimentos foram organizados em quatro fases, variando o número de *threads*, o escalonamento, a afinidade e a técnica de *first-touch*, a fim de observar o impacto de cada configuração em cada aplicação.

2 Aplicações escolhidas

Este trabalho utilizou o cálculo do conjunto de Mandelbrot para a experimentação em uma aplicação *compute-bound*, e o algoritmo Stencil 2D de Jacobi para avaliar as diferentes configurações em um problema *memory-bound*. As aplicações foram desenvolvidas com auxílio da ferramenta de inteligência

artificial Claude Opus 4.7 da empresa Anthropic. Além disso, todo o contexto do trabalho está disponível em um repositório no GitHub¹.

2.1 Conjunto de Mandelbrot

O Conjunto de Mandelbrot é um fractal de tempo de escape criado pelo matemático Benoit B. Mandelbrot [4]. Mandelbrot criou o termo fractal para descrever objetos que apresentam uma mesma estrutura em diferentes escalas [1]. Esse conjunto é definido pela Equação 1, onde a iteração inicia com $z_0 = 0$ e c é um número complexo no formato $c = x + yi$. Para cada valor de c a Equação 1 gera uma sequência de números complexos $(z_0, z_1, z_2, \dots)$. Se essa sequência não tende ao infinito, o ponto c pertence ao conjunto de Mandelbrot. Caso contrário, o ponto não faz parte do conjunto.

$$\begin{aligned} z_0 &= 0 \\ z_{n+1} &= z_n^2 + c \end{aligned} \tag{1}$$

A implementação calcula uma imagem de tamanho 4096×4096 pixels sobre a região $[-2, 2] \times [-2, 2]$ do plano complexo, com limite de 1000 iterações por pixel. Caso o ponto atinja esse limite, considera-se que ele não pertence ao conjunto. O Algoritmo 1 mostra a implementação.

¹As implementações, os resultados e os gráficos estão disponíveis em https://github.com/matrengnago/openmp_strategies.

Algorithm 1 Cálculo do conjunto de Mandelbrot

Require: dimensões $W \times H$, MAX_ITER

```
1: alocar vetor  $image[W \times H]$ 
2: for  $y \leftarrow 0$  to  $H - 1$  do
3:   for  $x \leftarrow 0$  to  $W - 1$  do
4:      $c \leftarrow$  mapeia  $(x, y)$  para o plano complexo
5:      $z \leftarrow 0$ ;
6:      $iter \leftarrow 0$ ;
7:     while  $|z| \leq 2$  and  $iter < MAX\_ITER$  do
8:        $z \leftarrow z^2 + c$ 
9:        $iter \leftarrow iter + 1$ 
10:    end while
11:     $image[y \times W + x] \leftarrow iter$ 
12:  end for
13: end for
```

A paralelização do algoritmo é feita a partir da diretiva `#pragma omp parallel for` sobre o laço externo que itera sobre as linhas da imagem. Como o cálculo de cada pixel é feito de forma independente, não há necessidade de sincronização.

2.2 Stencil 2D

O stencil define um padrão computacional onde os elementos de uma grade multidimensional são atualizados com base nos valores de um conjunto fixo de pontos vizinhos [3]. Computações baseadas em stencils são utilizadas em diversas aplicações científicas, como simulações de dinâmica de fluidos computacional e simulações de eletromagnetismo [5]. O Algoritmo 2 mostra a implementação de um stencil bidimensional de Jacobi [2]. A computação calcula a média aritmética de cada ponto da grade e de seus vizinhos imediatos em todas as direções. Essa implementação é composta por três laços aninhados: o laço mais externo itera sobre os passos de tempo, enquanto os dois laços internos percorrem toda a grade 2D.

Algorithm 2 Stencil 2D de Jacobi

Require: malha $N \times N$, número de iterações T

```
1: alocar vetores  $grid[N \times N]$  e  $new\_grid[N \times N]$ 
2: inicializar  $grid$  e  $new\_grid$ 
3: for  $k \leftarrow 0$  to  $T - 1$  do
4:   for  $i \leftarrow 1$  to  $N - 2$  do
5:     for  $j \leftarrow 1$  to  $N - 2$  do
6:        $new\_grid[i \times N + j] \leftarrow 0,25 \times ($ 
7:          $grid[(i - 1) \times N + j] + grid[(i + 1) \times N + j] +$ 
8:          $grid[i \times N + (j - 1)] + grid[i \times N + (j + 1)])$ 
9:     end for
10:  end for
11:  trocar ponteiros  $grid \leftrightarrow new\_grid$ 
12: end for
```

A paralelização desse algoritmo também usa a diretiva `#pragma omp parallel for` sobre o laço que itera sobre as linhas. Como a escrita de `new_grid` lê apenas `grid`, não há dependências entre *threads* dentro de uma iteração.

Além disso, a implementação possui dois modos possíveis para a inicialização das matrizes: o modo sequencial, em que a *thread* mestra toca todas as páginas, e o modo paralelo, com o escalonador `static`, em que cada *thread* toca as páginas que irá processar.

3 Metodologia experimental

Todos os experimentos foram executados na partição hype do PCAD², no nó `hype1`. Essa máquina possui uma placa-mãe HP ProLiant DL380 Gen9 com dois processadores Intel Xeon E5-2650v3 2,3GHz, totalizando 20 cores físicos, e 128GB de memória DDR4. A máquina expõe dois nós NUMA, um por processador.

As execuções foram submetidas por um job slurm. Além disso, todo o ambiente é fornecido de forma reprodutível por um flake Nix, que fixa as versões de todos os pacotes. Os dois programas foram compilados com GCC usando a flag `-O3` de otimização. O tempo de execução é medido pela função `omp_get_wtime()` e cada configuração foi repetida 10 vezes.

²<https://pcad.inf.ufrgs.br>

Os experimentos foram divididos em quatro fases. A Tabela 1 mostra os parâmetros utilizados em cada uma das fases.

Tabela 1: Fases do experimento.

Fase	Variáveis	Configuração
1	OMP_NUM_THREADS	{1, 2, 4, 8, 10, 16, 20, 32, 40, 64}.
2	OMP_SCHEDULE $\times$ <i>chunk</i>	{static, dynamic, guided, auto} {1, 2, 4, 8, 16, 32, 64}; fixo em 20 <i>threads</i> .
3	OMP_PROC_BIND OMP_PLACES	{true, false, master, close, spread} {sockets, cores, threads}; fixo em 20 <i>threads</i> .
4	FIRST_TOUCH OMP_NUM_THREADS	{0, 1} $\times$ {8, 16, 20, 32, 40}; apenas stencil; OMP_PROC_BIND=spread, OMP_PLACES=cores.

A primeira fase avaliou a escalabilidade e a eficiência paralela de ambos os algoritmos sob o aumento do número de *threads*. A segunda fase teve o foco em como o tempo de execução é afetado pelas diferentes políticas de escalonamento (OMP_SCHEDULE). A terceira fase se concentrou nas configurações de afinidade, visando a otimização da localidade dos dados. E por fim, a quarta e última fase foi dedicada exclusivamente ao Stencil, *memory-bound*, para analisar o impacto da técnica de *first-touch*. Essa etapa teve como objetivo verificar se a inicialização paralela diminui os custos de acesso à memória NUMA, garantindo que os dados sejam alocados fisicamente mais próximos aos cores que os processam.

4 Resultados obtidos

A partir dos experimentos descritos na seção anterior, a seguir, serão apresentados os resultados obtidos em cada uma das fases.

4.1 Escalabilidade

A Figura 1 apresenta o *speedup* e a eficiência paralela de ambos os algoritmos em função do número de *threads*, usando como referência o tempo da execução com uma thread.

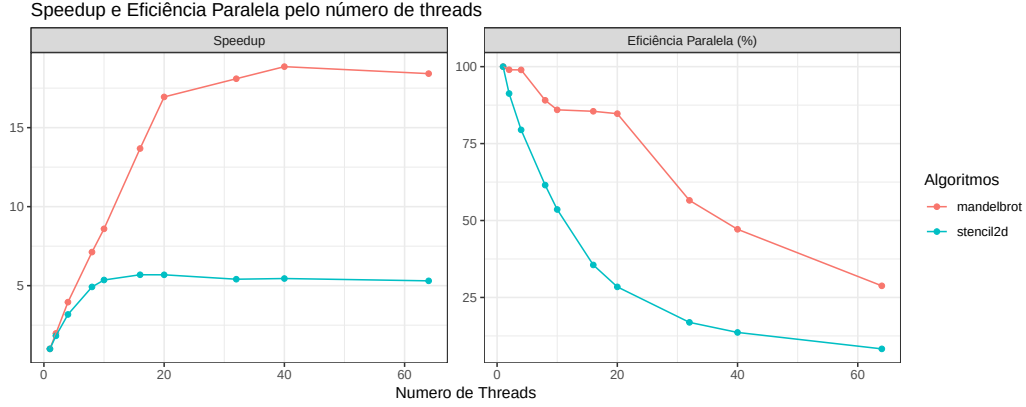


Figura 1: *Speedup* e eficiência paralela em função do número de *threads*.

O conjunto de Mandelbrot manteve a eficiência acima de 98% em até 4 *threads*. Com 20 *threads*, ocupando todos os cores físicos da máquina, o algoritmo atingiu um *speedup* de 16.9 com 84.7% de eficiência. Esse resultado surge pela natureza *compute-bound* do algoritmo, onde cada pixel é calculado de forma independente ou seja, a carga de trabalho depende muito da capacidade de processamento da CPU. A partir de 32 *threads*, quando o número de *threads* ultrapassa o número de cores físicos, o *speedup* se estabilizou próximo de 18.8, e a eficiência caiu drasticamente, refletindo a disputa de recursos pelas *threads* que compartilham o mesmo core físico.

No Stencil 2D a escalabilidade se manteve próxima do ideal até 4 *threads*, e a partir daí o *speedup* saturou em torno de 5.7 entre 16 e 20 *threads*. Como o algoritmo realiza pouco trabalho aritmético por elemento acessado, o desempenho passa a ser limitado pela velocidade da memória e não pela quantidade de cores disponíveis.

4.2 Escalonadores

Nesta fase, o número de *threads* foi fixado em 20 e variou-se a política de escalonamento e o tamanho do *chunk*. A Figura 2 apresenta a média do tempo de execução para cada combinação.

No Mandelbrot, a política de escalonamento que apresentou o melhor resultado foi a **dynamic** com 2 *chunks*, atingindo 1.23s. Esse resultado ficou bem próximo das políticas **static** e **auto**. Isso ocorre porque a carga de trabalho é desbalanceada, onde os pixels que pertencem ao conjunto consomem

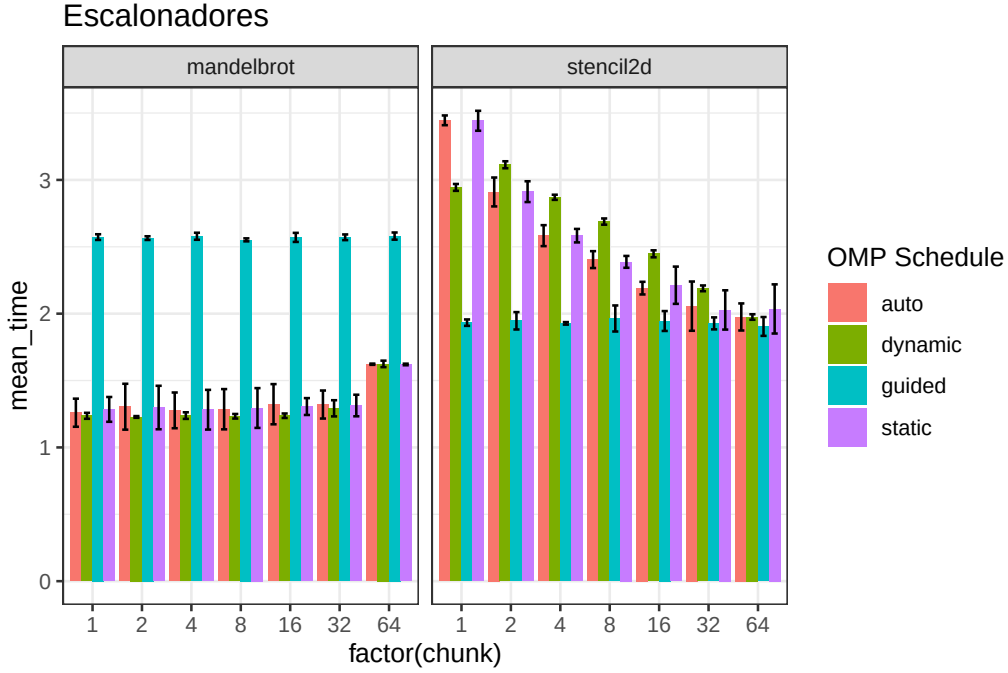


Figura 2: Tempo médio de execução em função da política de escalonamento e do tamanho do *chunk*.

o número máximo de iterações e ficam alocados na região do centro da imagem. A política *guided*, que distribui *chunks* grandes no início da execução foi a pior de todas, com tempo de execução em torno de 2.57s, porque esses blocos iniciais grandes geram desbalanceamento entre as *threads*. Além disso, *chunks* muito grandes, tamanho 64 por exemplo, afetam o desempenho de todas as políticas, pelo mesmo motivo.

Porém, o Stencil 2D apresentou um comportamento inverso. Por ter uma carga praticamente homogênea, ele se beneficiou de *chunks* grandes, que preservam a localidade dos acessos à memória e reduzem o número de blocos distribuídos. A política *guided* se apresentou como a melhor independentemente do tamanho do *chunk*, enquanto *static*, *dynamic* e *auto* melhoram à medida que o tamanho do *chunk* aumenta. Com *chunk* baixo, as linhas da cache são distribuídas de forma intercalada entre as *threads*, o que prejudica a localidade espacial e acaba afetando negativamente o Stencil 2D, que é uma aplicação *memory-bound*.

4.3 Afinidade

A terceira fase, também foi executada com 20 *threads* fixas, visa testar e observar o efeito das políticas de afinidade combinando os valores de `OMP_PROC_BIND` e `OMP_PLACES`. A Figura 3 mostra o tempo de execução médio de cada dupla.



Figura 3: Tempo médio de execução para cada combinação de `OMP_PROC_BIND` e `OMP_PLACES`, com 20 *threads*. Tons mais escuros indicam tempos menores.

O resultado mais interessante é o impacto da política **master**, que fixa todas as *threads* no mesmo *place* da *thread* mestra, fazendo com que todas disputem os recursos desse único *place*. A localidade influenciou consideravelmente o tempo de execução nessa política. Com a localidade **threads**, todas as *threads* competem pela mesma *thread* de *hardware*, resultando no pior tempo em ambos os algoritmos, com valores parecidos com a execução sequencial. Com **cores**, o desempenho também é fortemente prejudicado, com a diferença de que as *threads* disputam o mesmo core físico. Já com **sockets**, todas as *threads* ficam presas a um único processador. Nessa localidade, a execução é mais lenta que nas políticas que usam os dois processadores, mas ainda assim aproveita todos os cores de um *socket*, o que explica o ganho expressivo em relação às localidades **cores** e **threads**.

É possível observar outro resultado interessante ao combinar a localidade **threads** com as políticas que empacotam as *threads*, **close** e **true**. Como cada *place* passa a ser um único contexto lógico, o empacotamento aloca as 20 *threads* nos primeiros 20 *places* disponíveis, que correspondem às 20 *threads* lógicas dos 10 cores físicos do primeiro *socket*. Assim, cada núcleo físico passa a hospedar duas *threads*, enquanto o segundo *socket* permanece ocioso, o que ocasiona em um tempo de execução dobrado. Em contrapartida, o mesmo **OMP_PLACES=threads** junto com a política **spread** não apresenta esse problema, porque o **spread** distribui as *threads* pelos dois *sockets* antes de recorrer às *threads* lógicas. As demais combinações apresentam desempenho muito parecido, indicando que, uma vez garantida a ocupação dos dois *sockets* e de um core físico por *thread*, a política específica de afinidade tem pouca influência sobre o tempo de execução dos algoritmos.

4.4 First-touch

A última fase foi dedicada ao Stencil, *memory-bound*, e comparou a inicialização sequencial das matrizes, em que a *thread* mestra toca todas as páginas, com a inicialização paralela, em que cada *thread* toca as páginas que irá processar. Todas as execuções utilizaram **OMP_PROC_BIND=spread** e **OMP_PLACES=cores**, variando o número de *threads*. A Figura 4 apresenta os tempos médios obtidos.

A inicialização paralela é consistentemente mais rápida do que a sequencial, com ganhos entre 13% e 19% em todas as contagens de *threads*. O melhor caso ocorre com 20 *threads*, em que o tempo cai de 3.45s para 2.91s. Esse resultado confirma o efeito da política de *first-touch* em arquiteturas NUMA, onde ao tocar cada página pela primeira vez a partir da *thread* que a utilizará, as páginas são alocadas fisicamente no nó NUMA do processador correspondente, evitando os acessos remotos que ocorrem quando toda a matriz é inicializada pela *thread* mestra e fica concentrada em um único nó.

5 Conclusões

Os experimentos mostraram que não existe uma única configuração ideal para todos os casos. A melhor escolha depende do tipo de aplicação. A diferença entre um programa *compute-bound* e um *memory-bound* ficou clara em todas as fases.

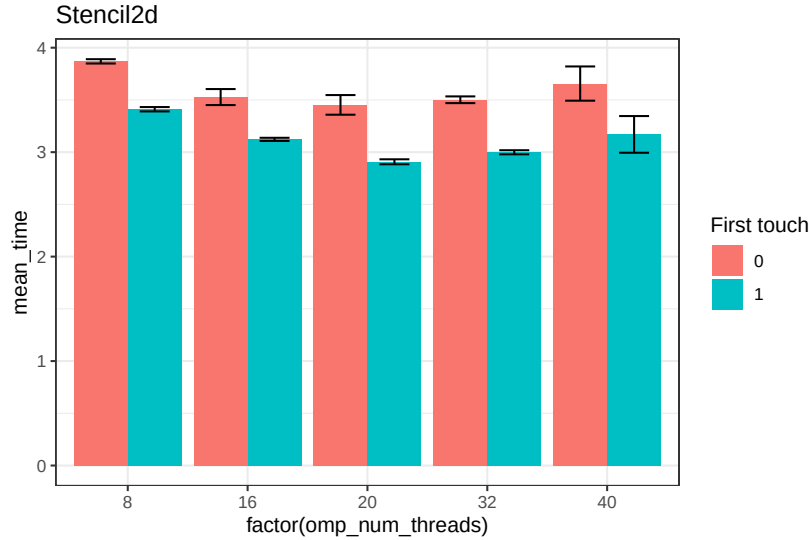


Figura 4: Tempo médio de execução do Stencil 2D com inicialização sequencial (`first_touch=0`) e paralela (`first_touch=1`), em função do número de *threads*. As barras de erro indicam o desvio padrão.

O Mandelbrot, por ser *compute-bound*, aproveitou bem o aumento do número de *threads* e escalou quase de forma ideal enquanto havia cores físicos disponíveis, chegando a um *speedup* próximo de 17 com 20 *threads*. Por ter uma carga de trabalho desbalanceada, ele se beneficiou de um escalonamento *dynamic* com *chunks* pequenos, que distribui melhor as linhas mais custosas.

O Stencil 2D, por ser *memory-bound*, teve um comportamento bem diferente. Ele parou de escalar cedo, por volta de 4 a 8 *threads*, pois ficou limitado pela largura de banda da memória. No escalonamento, preferiu *chunks* grandes, que preservam a localidade dos dados, sendo a política *guided* a mais eficiente. Além disso, usar a técnica de *first-touch*, inicializando os dados em paralelo, reduziu o tempo de execução, ao manter os dados próximos dos cores que os utilizam na arquitetura NUMA.

Referências

- [1] Thiago Albuquerque Assis. *Geometria fractal: propriedades e características de fractais ideais*. Revista Brasileira de Ensino de Física, 2008.

- [2] J.W. Demmel. *Applied Numerical Linear Algebra*. Other Titles in Applied Mathematics. Society for Industrial and Applied Mathematics (SIAM, 3600 Market Street, Floor 6, Philadelphia, PA 19104), 1997.
- [3] Alain Denzler, Rahul Bera, Nastaran Hajinazar, Gagandeep Singh, Geraldo Oliveira, Juan Gómez-Luna, and Onur Mutlu. Casper: Accelerating stencil computation using near-cache processing. 12 2021.
- [4] Benoit B. Mandelbrot. *The Fractal Geometry of Nature*. W. H. Freeman and Company, New York, 1982.
- [5] Allen Taflove. Review of the formulation and applications of the finite-difference time-domain method for numerical modeling of electromagnetic wave interactions with arbitrary structures. *Wave Motion*, 10(6):547–582, 1988. Special Issue on Numerical Methods for Electromagnetic Wave Interactions.